■ Google Meet Video Recording Script & Defense Guide

DISTRIBUTED AIR QUALITY ANALYTICS & FORECASTING (TRACK B) • NATURAL SPEAKING FLOW FOR SIR

How to Record on Google Meet: One teammate shares their screen with the terminal open in the commands/ folder. The driver runs each numbered script (./1_hdfs_show_raw.sh, etc.). When the command finishes and output appears on screen, the assigned teammate speaks naturally to Sir, explaining the exact numbers and figures displayed.

Speaker	Role / Layer	Assigned Commands	What They Cover in the Video
Teammate A	Driver / HDFS Storage Lead	./1_hdfs_show_raw.sh ./2_hdfs_show_parquet.sh	Video intro, 96k raw data in HDFS, Parquet lake & partition pruning
Teammate B	Spark ETL & Analytics Lead	./3_spark_run_etl.sh ./4_spark_run_analytics.sh	Spark ETL, CPCB 16h completeness rule, Delhi vs Bengaluru, diurnal peaks
Teammate C	Spark MLlib Machine Learning Lead	./5_spark_mllib_forecasting.sh ./6_spark_mllib_classification.sh	Chronological split, Random Forest 24h forecast (+16.6%), 70.2% hazard classifier

Opening Speech (30s) — Teammate A (Screen Sharer):
*"Hello Sir. In this video, our team is presenting our Big Data project: **Distributed Urban Air Quality Analytics and AQI Forecasting**. We have built this project according to **Track B: Machine Learning at Scale**, implementing the complete distributed pipeline: **Hadoop HDFS** → **Apache Spark distributed ETL** → **Spark MLlib**. We are analyzing continuous sensor telemetry across three cities—**Delhi NCR**, **Bengaluru**, and **Hyderabad**—adhering strictly to India's official **Central Pollution Control Board (CPCB)** regulatory standards. We have prepared numbered commands so we can run and demonstrate each stage cleanly. Let us begin with Step 1."*

Step 1: Inspect Raw Telemetry in Hadoop HDFS — Teammate A (Driver / Storage Lead)

Terminal Action: Driver runs ./1_hdfs_show_raw.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ hdfs dfs -ls -h /air_quality/raw/`
`Found 1 items`
`-rw-r--r--  1 yash supergroup  47.3 M`
`  /air_quality/raw/`
`  raw_hourly_measurements.jsonl`
	What Teammate A (Driver / Storage Lead) Says to Sir Naturally:
	"Sir, as you can see on the screen, our command just listed our raw telemetry staged inside Hadoop HDFS under the path /air_quality/raw/. You can see this file: raw_hourly_measurements.jsonl. It is 47.3 Megabytes and contains 96,000 raw continuous hourly records collected from the OpenAQ API. We built a resumable client that respects OpenAQ's 60 requests-per-minute rate limit. Staging it in HDFS allows Hadoop to distribute the data with a 128 MB block size and 3x replication factor. Now, let us look at Step 2 to see how we store the processed data."

Step 2: Inspect Partitioned Parquet Lake in HDFS — Teammate A (Driver / Storage Lead)

Terminal Action: Driver runs ./2_hdfs_show_parquet.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ hdfs dfs -ls -h .../hourly_aqi_parquet/`
`Found 4 items`
`_SUCCESS`
`city=Bengaluru`
`city=Delhi NCR`
`city=Hyderabad`
	What Teammate A (Driver / Storage Lead) Says to Sir Naturally:
	"Sir, in Step 2, this is our processed data lake in HDFS. Notice that instead of plain CSV files, we are using columnar Apache Parquet with Snappy compression, partitioned by city—Bengaluru, Delhi NCR, and Hyderabad. This enables Partition Pruning: when our downstream Spark queries ask for Delhi data, Spark reads only the Delhi directory and completely skips the other cities, saving disk I/O and network shuffle. Now my teammate will explain Step 3 and run our distributed Spark ETL."

Step 3: Execute Distributed Apache Spark ETL — Teammate B (Spark ETL & Analytics Lead)

Terminal Action: Driver runs ./3_spark_run_etl.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ python3 scripts/spark_etl.py`
`================ STARTING SPARK ETL ================`
`[1/6] Ingested 96,000 raw records.`
`[3/6] Normalizing IST & units (ppb -> ug/m3)`
`[4/6] Pivoting into hourly station records...`
`[5/6] Total Station-Hours Processed: 18,521`
`Records Meeting CPCB 16h Rule: 12,792`
`Sample Processed Records:`
`Delhi NCR | R K Puram | cpcb_aqi: 500.0 (Severe)`
`Spark ETL completed in 20.71 seconds.`
	What Teammate B (Spark ETL & Analytics Lead) Says to Sir Naturally:
	"Sir, as you can see on screen, Spark 4.2.0 executed our ETL pipeline in 20 seconds. It loaded the 96,000 raw records and pivoted them into 18,521 station-hour records. Here in step 5, our code enforces the official Indian CPCB 16-hour completeness rule: a 24-hour moving window is only legally valid if at least 16 hourly readings exist. Exactly 12,792 records met this strict rule. In the sample table, you can see the calculated 24h PM2.5, PM10, and overall AQI—for example, R K Puram in Delhi has an AQI of 500, which falls into the Severe category. Now let us run Step 4."

Step 4: Run Distributed Spark SQL Analytics & Visual Trends — Teammate B (Spark ETL & Analytics Lead)

Terminal Action: Driver runs ./4_spark_run_analytics.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ python3 scripts/run_analytics.py`
`[1/4] City Statistics:`
`Delhi: avg PM2.5 = 121.9 | avg AQI = 382.0`
`Bengaluru: avg PM2.5 = 28.8 | avg AQI = 293.9`
`Hyderabad: avg PM2.5 = 28.0 | avg AQI = 265.1`
`[2/4] Hazard Distribution:`
`Delhi Severe hours: 62.5%`
`Bengaluru Satisfactory: 36.6%`
`[4/4] Saved:`
`[CMD 4 - FIG 1] cmd4_fig1_diurnal_pm25_trends.png`
`[CMD 4 - FIG 2] cmd4_fig2_cpcb_category_distribution.png`
`[CMD 4 - FIG 3] cmd4_fig3_city_aqi_boxplots.png`
	What Teammate B (Spark ETL & Analytics Lead) Says to Sir Naturally:
	"Sir, our Spark SQL aggregation is now displayed on screen. Looking at Table 1, Delhi's average PM2.5 is 121.9 ug/m3—over four times higher than Bengaluru (28.8). In Table 2, 62.5% of all recorded hours in Delhi fell into the Severe category. Below the tables, Spark generated three labeled figures: Figure CMD 4-1 shows the diurnal cycle—there is a huge peak at 8:00 AM due to morning thermal inversion trapping smoke, and another peak at 8:00 PM during rush hour. Figures 4-2 and 4-3 show category distributions and AQI boxplots. Now my teammate will present Step 5 for MLlib."

Step 5: Train Distributed Spark MLlib 24-Hour Forecasting — Teammate C (Spark MLlib Machine Learning Lead)

Terminal Action: Driver runs ./5_spark_mllib_forecasting.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ python3 scripts/train_forecasting_models.py`
`[2/5] Chronological Split (80% Train, 20% Test)`
`[3/5] Naive Persistence Baseline -> RMSE: 42.16`
`[5/5] Training MLlib Distributed Models:`
`- Linear Regression: RMSE 37.86 (+10.2%)`
`- Random Forest: RMSE 35.17 (+16.6%)`
`- GBT Regressor: RMSE 37.61 (+10.8%)`
`Best Model: Random Forest Regressor (RMSE 35.17)`
`Saved: [CMD 5 - FIG 1] cmd5_fig1_model_forecast...`
	What Teammate C (Spark MLlib Machine Learning Lead) Says to Sir Naturally:
	"Sir, now we move to distributed Machine Learning using Spark MLlib to forecast PM2.5 levels 24 hours in advance. In Step 2, we used a strict chronological 80/20 block split—the model trains strictly on the past and tests on the future so there is zero data leakage. In Step 3, our Persistence Baseline gave an RMSE of 42.16. Then Spark trained three models. Our champion model is Random Forest with an RMSE of 35.17—a 16.6% improvement over the baseline in just 1.79s of training. It saved Figure CMD 5-1, where you can see the orange predicted curve closely tracking the actual observed values. Next is Step 6."

Step 6: Train Distributed Spark MLlib 6-Tier Hazard Classifier — Teammate C (Spark MLlib Machine Learning Lead)

Terminal Action: Driver runs ./6_spark_mllib_classification.sh in terminal.

Exact Screen Output (What Sir Sees)	What to Say While Displaying This Output

`$ python3 scripts/train_classification_model.py`
`Training distributed RandomForestClassifier...`
`--- Spark MLlib Classification Metrics ---`
`Accuracy: 70.21%`
`Weighted F1-Score: 0.676`
`Weighted Precision: 0.674`
`Weighted Recall: 0.702`
`Training Duration: 3.60s`
`Saved: [CMD 6 - FIG 1] cmd6_fig1_cpcb_classification ...`
	What Teammate C (Spark MLlib Machine Learning Lead) Says to Sir Naturally: "Sir, this is our second ML model for classification. Instead of just numbers, it predicts the exact CPCB health category 24 hours ahead—Good, Satisfactory, Moderate, Poor, Very Poor, or Severe. Looking at the metrics on screen, the Random Forest Classifier achieved 70.21% test accuracy and a weighted F1-score of 0.676 in just 3.6 seconds. It saved the confusion matrix as CMD 6-1, showing how reliably it identifies Severe and Satisfactory days. Driver, please show Step 7."

Step 7: Master Automated Pipeline & Closing (1 Minute)

Terminal Action: Driver runs **./7_run_entire_pipeline.sh** (or points to it).
What Teammate A Says to Conclude the Video:
*"And Sir, if we ever want to run the entire distributed pipeline end-to-end automatically—from data validation and Spark ETL to analytics and all MLlib models—we have **./7_run_entire_pipeline.sh**, which executes everything in one automated run.*

To summarize our project:
*1. We built a full **Track B** distributed architecture on **HDFS, Apache Spark, and Spark MLlib**.*
*2. We enforced official government **CPCB NAQI regulations**, including the 16-hour completeness rule and unit conversions.*
*3. We proved machine learning at scale with a **16.6% improvement in 24h forecasting** and **70.2% hazard classification accuracy**.*

Thank you, Sir!"

■■ Quick Figure Guide for Video Screen Display

If you open the generated images in reports/figures/ during the video, say this:

Command	Generated Image File	Exact Sentence to Say to Sir while Showing Image
Command 4	cmd4_fig1_diurnal_pm25_trends.png	"Sir, look at this spike in Delhi at 8:00 AM due to winter thermal inversion trapping pollutants, and the evening spike at 8:00 PM from traffic."
Command 4	cmd4_fig2_cpcb_category_distribution.png	"Notice the tall dark red bar for Delhi showing 62.5% Severe hours, while Bengaluru is mostly green and orange."
Command 4	cmd4_fig3_city_aqi_boxplots.png	"This boxplot shows the median AQI in Delhi is right near the 400 Severe threshold, whereas Bengaluru and Hyderabad stay much lower."
Command 5	cmd5_fig1_model_forecast_actual_vs_pred.png	"Here the orange predicted line tracks the actual blue observed curve accurately across the 24-hour future test horizon."
Command 6	cmd6_fig1_cpcb_classification_confusion_matrix.png	"The heatmap diagonal is dark, proving the classifier reliably identifies Severe days without confusing them with cleaner categories."